

result to the `parse` method. This is where all the magic will happen. This is where we will use our selectors to extract data and return it to Scrapy. Let's look at the completed `parse` method:

```
def parse(self, response):
    repos = response.selector.css('ol.repo-list li')
    for repo in repos:
        name = repo.xpath('://h3/a/@href').extract_first()
        lang = repo.css(
            'span[itemprop="programmingLanguage"]::text'
        ).extract()
        stars = repo.css(
            'a[aria-label="Stargazers"]::text'
        ).extract()
        forks = repo.xpath(
            '://a[@aria-label="Forks"]/text()'
        ).extract()
        yield GithubItem(
            repo=name,
            language=lang,
            stars=stars,
            forks=forks,
        )
```

Here we are making a selection of all the trending repositories and then extracting details from them one by one. After extracting the data, we are wrapping it into a `GithubItem` and yielding that. (You will need to add `from github.items import GithubItem` at top of the file to use `GithubItem`)

The difference between `return` and `yield` is something that often confuses people. Look at it as a way for a function to return multiple values, one by one as they are computed. The function can 'return' a value and keep running where it left off. It would be useful to read up about `yield` as it is a useful construct and has now been adopted by JavaScript as well.

If while scraping you discover another URL you want to scrape, you can yield a request (using `scrapy.Request(some_url)`) instead of an item Scrapy would automatically add that to the list of URLs to fetch.

Sometimes these URLs could be entirely tangential to your current scrape and require a different parsing strategy. In that case, you can provide the alternative parse function for Scrapy to use with that URL. Here is how you could tell Scrapy to use the `parse2` method to parse the data fetched from `some_url`: `yield scrapy.Request(some_url, callback=self.parse2)`

If you want the second parsing method to have access to the item so that it can add the data it scrapes to the existing item, you can do that as well, but you will need to explicitly include that in the request. Here is how you can do that:

```
yield scrapy.Request(details_url,
    callback=self.parse_details,
    meta={'item': item})
```

Here Scrapy will fetch the `details_url` and pass it to the `parse_details` method. In that method, we will be able to access the item data by accessing `response.meta['item']`.

You can now run this using the `scrapy crawl` command:

```
scrapy crawl trending -o trending.csv
```

The above command instructs Scrapy to start scraping using the 'trending' spider; and the `-o` lets is specify the output file

name. Here the output will be in the CSV format based on the file extension. JSON and some other formats are also supported.

Note that Scrapy will not delete existing data if you run the command multiple times; new data will be added to the end. This might not be what you want.

If you open the CSV file now, you'll see that all the data we wanted is there, but it's a little messy with extra whitespaces. Let's use a pipeline to get rid of that.

Adding a Pipeline

If you open `pipelines.py` you will find that Scrapy has already created a basic pipeline class for you. This class has a single method `process_item` that simply returns the item it gets as is. In this method we can add out logic to clean up the item. Here is our final pipeline class:

```
class GithubPipeline(object):
    def process_item(self, item, spider):
        item['language'] = ".join(item['language']).strip()
        item['stars'] = ".join(item['stars']).strip()
        item['forks'] = ".join(item['forks']).strip()
        return item
```

Since `extract` returns a list of strings, here we just join those strings into a single string, and strip the whitespace padding. This could be done in the Spider, but pipelines are easier to reuse and often more complex than the example above.

Scrapy won't automatically use this pipeline, you'll need to enable it in `settings.py`. A section for that already exists, but is commented out. Uncomment the `ITEM_PIPELINES` declaration and change the name of the pipeline to match our pipeline name. It should look like this:

```
ITEM_PIPELINES = [
    'github.pipelines.GithubPipeline': 300,
]
```

The numerical value here is used to set the order in which pipelines are used, in case there is more than one. If you run the scraper now, you will find the output is cleaned up.

Next Steps

There is a whole lot more you can do with Scrapy by taking advantage of more complex concepts like middleware (which allows extending the functionality of Scrapy), and signals (which notify you of scraping events). Just exploring the settings file should give you an idea of where you can go next.

You can also look into `scrapyd`, a Python package that turns your scraper into a web service so you can start crawls, check status of crawls and get crawl results using a web API. This can be useful to integrate the scraper with code in another language. ScrapingHub is a service that provides hosting for Scrapy spiders, and has a free tier.

The major task, no matter what tool you use for scraping will always be able to identify how and where the data you want it available, and how you can extract it. This is a skill that you can hone with practise as you encounter different kinds of websites that challenge you in different ways.

Your next step should be to check out the Scrapy documentation to learn many of the things we haven't covered here.

NOTE: You can find the complete code for this project on GitHub at: <http://dgit.in/WebScrpngDgt>